



PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering

Software Requirements Specification (SRS)

AttendEase – Student Attendance Management System Team 3 | Course UE24CS341A | PES University

Revision History

Version	Date	Description	Author(s)
1.0	02 September 2026	Initial draft.	Team 3
1.1	03 September 2026	Regrouped System Features into 4 team ownership roles; added FR-037/FR-038.	Team 3
1.2	03 October 2026	Added UML Use Case diagram (Fig. 1) to Section 4.1; split Section 6.3 into Security Objectives and Security Requirements.	Team 3

Approvals

Name	USN	Date
Arundhati K	PES1UG24CS083	03-Oct-2026
Anshdeep Singh Sachdeva	PES1UG24CS070	03-Oct-2026
Atharva Ashish Vyas	PES1UG24CS093	03-Oct-2026
Bhoomika Dayanand Jituri	PES1UG25CS807	03-Oct-2026

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements of AttendEase, a web-based Student Attendance Management System developed by Team 3 for the PES University Software Engineering Level 3 Mini-Project (Course UE24CS341A). It is intended to serve as the authoritative reference for design, implementation, and testing of the system.

1.2 Intended Audience

- Team 3 developers implementing AttendEase.
- Course instructor and evaluators assessing the mini-project deliverables.
- Future maintainers or reviewers of the system.

1.3 Product Scope

AttendEase allows administrators to configure courses, sections and faculty assignments; faculty to record attendance per session and manage correction requests; and students to view their attendance and raise correction requests. The system targets a single-department academic deployment.

1.4 References

- PES University SE Level 3 Mini-Project Guidelines and Deliverables Checklist.
- IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications.
- AttendEase Project Synopsis, Team 3.
- Coffee Kiosk Example — SE Level 3 Project Submission Guide (reference exemplar).

2. Overall Description

2.1 Product Perspective

AttendEase is a new, self-contained web application. It is not a modification of an existing system. It integrates with an SMTP email service for correction-request notifications but otherwise operates as a standalone system.

2.2 Product Functions

- User registration, login and role-based access control.
- Course, section and timetable management.
- Faculty-to-section assignment.
- Attendance session creation and per-student attendance marking.
- Student attendance dashboard.
- Attendance correction request submission and approval workflow.
- Attendance shortage / defaulter identification.
- Attendance report generation.

2.3 User Classes and Characteristics

User Class	Characteristics
Student	Views own attendance; raises correction requests. Assumed basic web-literacy.

User Class	Characteristics
Faculty	Marks attendance; reviews and approves/rejects correction requests; generates reports for their sections.
Administrator	Configures courses, sections, and faculty assignments; has system-wide visibility.

2.4 Operating Environment

- Server: Python 3.11 / Django 4.x application server.
- Database: PostgreSQL 14+.
- Frontend: Modern web browsers (Chrome, Firefox), Bootstrap 5 UI.
- Deployment: Docker containers, GitHub Actions CI/CD.

2.5 Design and Implementation Constraints

- Must be implemented in Python/Django within one academic semester.
- Must use Git/GitHub with mandatory pull-request-based code review.
- Must support role-based access for exactly three user classes (Student, Faculty, Administrator).
- Must be deployable via Docker for grading/demo purposes.
- Team of 4 members, each owning one of four defined roles.

2.6 Assumptions and Dependencies

- Each student and faculty member has a unique, pre-verifiable email address for account registration.
- Attendance is recorded per scheduled session, not continuously.
- The institution's academic calendar (courses, sections) is set up by an administrator before each semester.
- An SMTP relay is available for correction-request notification emails.
- The system is deployed for a single department/institution instance.

3. External Interface Requirements

3.1 User Interfaces

Three role-specific web interfaces (Student, Faculty, Administrator), built with Django templates and Bootstrap 5, accessible via standard web browsers.

3.2 Software Interfaces

PostgreSQL database via Django ORM; SMTP service via Django's email backend for correction-request notifications.

3.3 Communications Interfaces

HTTPS for all client-server communication; RESTful JSON API between the frontend and backend.

4. Analysis Models

4.1 Use Cases

ID	Use Case	Primary Actor(s)
UC-01	Register / Login	Student, Faculty, Administrator
UC-02	Manage Course & Timetable	Administrator
UC-03	Assign Faculty to Subject-Section	Administrator
UC-04	Start & Mark Attendance Session	Faculty
UC-05	View Attendance Dashboard	Student
UC-06	Raise Correction Request	Student
UC-07	Approve/Reject Correction Request	Faculty
UC-08	Generate Defaulter/Attendance Report	Faculty, Administrator

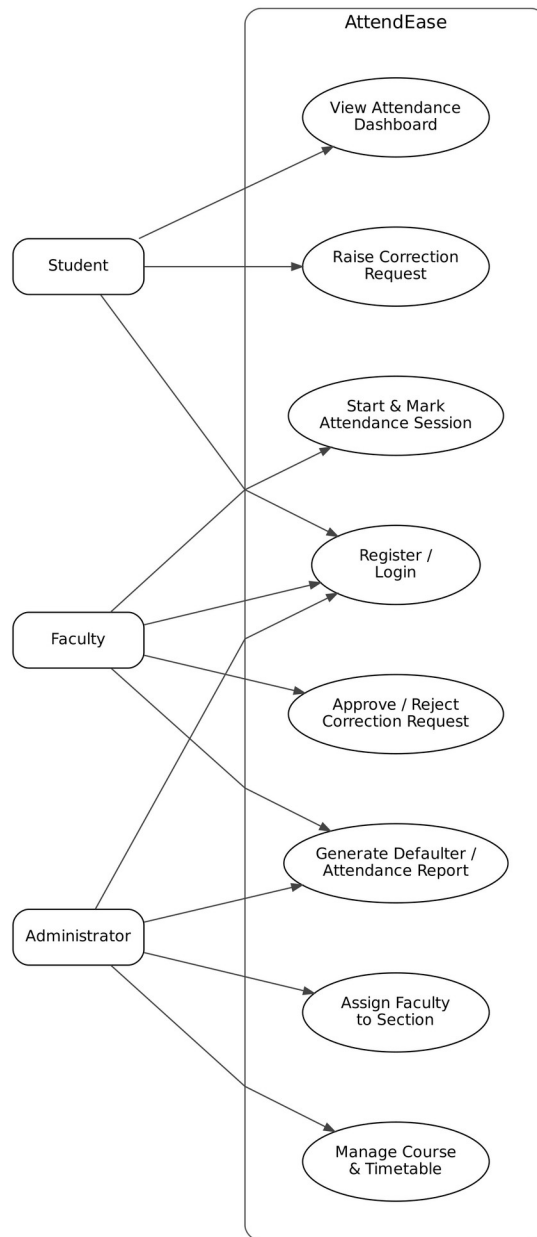


Figure 1: UML Use Case diagram for AttendEase, showing Student, Faculty and Administrator actors against use cases UC-01–UC-08.

4.2 Domain / Entity Model

The core domain entities are User (with role Student/Faculty/Administrator), Course, Section, Timetable, AttendanceSession, AttendanceRecord, and CorrectionRequest. A User with role Faculty is assigned to one or more Sections; each AttendanceSession belongs to a Section and produces one AttendanceRecord per enrolled Student; a CorrectionRequest references exactly one AttendanceRecord and transitions through Pending → Approved/Rejected states.

5. System Features

5.1 Role 1 — User Access & Student Portal

5.1.1 Authentication & Role-Based Access

Req ID	Requirement	Priority
FR-001	The system shall allow a new user to register with name, email, and role.	High
FR-002	The system shall verify email uniqueness at registration.	High
FR-003	The system shall allow a registered user to log in with email and password.	High
FR-004	The system shall enforce role-based access control for every protected endpoint.	High
FR-005	The system shall allow a user to log out, invalidating their session/token.	Medium
FR-034	The system shall allow a user to reset a forgotten password via email verification.	Medium

5.1.2 Student Attendance Dashboard

Req ID	Requirement	Priority
FR-020	The system shall display a student's attendance percentage per enrolled course.	High
FR-021	The system shall display a session-wise attendance history for a student.	Medium
FR-022	The system shall visually flag courses where attendance is below the shortage threshold.	Medium
FR-023	The system shall allow a student to filter attendance history by date range.	Low

5.2 Role 2 — Academic & Attendance Setup

5.2.1 Student & Academic Management

Req ID	Requirement	Priority
FR-006	The system shall allow an administrator to create and edit courses.	High
FR-007	The system shall allow an administrator to create and edit sections within a course.	High
FR-008	The system shall allow an administrator to enroll students into a section.	High
FR-009	The system shall allow an	Medium

Req ID	Requirement	Priority
	administrator to define a weekly timetable per section.	
FR-010	The system shall allow bulk student enrollment via CSV upload.	Low
FR-011	The system shall prevent duplicate section codes within a course.	Medium
FR-012	The system shall allow an administrator to deactivate a course or section.	Low
FR-013	The system shall allow an administrator to assign a faculty member to a section.	High

5.2.2 Attendance Recording

Req ID	Requirement	Priority
FR-014	The system shall allow faculty to start an attendance session for a scheduled section period.	High
FR-015	The system shall display the enrolled-student roster for the active session.	High
FR-016	The system shall allow faculty to mark each student present or absent.	High
FR-017	The system shall allow faculty to bulk-mark all students present with a single action.	Medium
FR-018	The system shall allow faculty to close/finalize a session after marking.	Medium
FR-019	The system shall timestamp each attendance session on creation and closure.	Low

5.3 Role 3 — Attendance Updates & Corrections

5.3.1 Attendance Management

Req ID	Requirement	Priority
FR-037	The system shall prevent duplicate attendance records for the same student and session.	High
FR-038	The system shall allow faculty to edit an attendance record within a configurable edit window after session closure.	Medium

5.3.2 Attendance Correction Requests

Req ID	Requirement	Priority
FR-024	The system shall allow a student to raise a correction request against a	High

Req ID	Requirement	Priority
	specific attendance record, with a reason.	
FR-025	The system shall notify the relevant faculty member by email when a correction request is raised.	Medium
FR-026	The system shall display a queue of pending correction requests to the relevant faculty member.	High
FR-027	The system shall allow faculty to approve a correction request, updating the underlying attendance record.	High
FR-028	The system shall allow faculty to reject a correction request with a reason.	Medium

5.4 Role 4 — Attendance Analytics & Reporting

5.4.1 Attendance Shortage Identification

Req ID	Requirement	Priority
FR-029	The system shall compute each student's attendance percentage per course.	High
FR-030	The system shall identify students below a configurable attendance-shortage threshold.	High
FR-035	The system shall allow the shortage threshold to be configured per course by an administrator.	Medium

5.4.2 Attendance Reports

Req ID	Requirement	Priority
FR-031	The system shall generate a defaulter report listing students below the shortage threshold for a section.	High
FR-032	The system shall generate a section-wise attendance summary report.	Medium
FR-033	The system shall allow reports to be exported as CSV/PDF.	Medium
FR-036	The system shall allow faculty/administrators to filter reports by date range and section.	Low

6. Other Nonfunctional Requirements

6.1 Performance

Req ID	Requirement
NFR-001	95% of API requests shall complete within 2 seconds under

Req ID	Requirement
	expected concurrent load (up to 50 simultaneous users).
NFR-002	The attendance-marking roster shall load within 3 seconds for a section of up to 80 students.

6.2 Safety

Req ID	Requirement
NFR-003	The system shall not permanently delete attendance records; corrections shall be tracked as an auditable change, not a destructive overwrite.
NFR-004	The system shall validate all session/date inputs to prevent attendance being recorded for future dates.

6.3 Security

6.3.1 Security Objectives

The following objectives guide the security design of AttendEase (elaborated further in the Software Architecture & Design Specification, Section 3.9):

Obj ID	Security Objective
SEC-OBJ-01	Ensure that only authenticated users with the correct role can access or modify attendance data (confidentiality and access control).
SEC-OBJ-02	Ensure that every change to an attendance record is attributable and auditable, so that correction-request decisions cannot be repudiated (integrity and accountability).
SEC-OBJ-03	Protect user credentials and session tokens in storage and in transit against disclosure or tampering (data protection).

6.3.2 Security Requirements

Req ID	Requirement
SEC-001	The system shall store all passwords using a salted, one-way hash (never in plaintext).
SEC-002	The system shall lock out or throttle login attempts after repeated failures on an account.
SEC-003	The system shall enforce server-side role-based authorization checks on every write endpoint, independent of client-side UI restrictions.
SEC-004	The system shall scope all attendance queries to the authenticated user's own role and enrolled/assigned sections, preventing cross-section data access.
SEC-005	The system shall require all client-server communication to occur over HTTPS.
SEC-006	The system shall record an audit trail (acted_by, acted_at) for every correction-request approval or rejection.

6.4 Software Quality Attributes

Req ID	Requirement
NFR-005	The system shall be maintainable: each of the four role-owned modules shall be independently testable.
NFR-006	The system shall achieve at least 70% automated test coverage on service-layer business logic by final submission.
NFR-007	The system's UI shall be usable on both desktop and tablet screen widths.
NFR-008	The system shall be deployable via a single docker-compose command for grading/demo purposes.

6.5 Business Rules / Domain Rules

- A student may raise at most one open correction request per attendance record at a time.
- An attendance session, once closed, may only be edited within the configurable edit window (FR-038).
- A faculty member may only mark attendance for sections they are assigned to.
- The attendance-shortage threshold defaults to 75% unless overridden per course by an administrator.
- A correction request, once Approved or Rejected, is immutable.
- Only an administrator may deactivate a course or section.

7. Other Requirements

- The system's database schema shall support future extension to multiple departments without a breaking migration.
- The system shall comply with the institution's data-retention and academic-record-keeping policies.
- Where feasible, the system shall reuse existing open-source Django packages (e.g., djangorestframework-simplejwt) rather than custom-building standard functionality.

Appendix A: Glossary

Term	Definition
Section	A specific offering of a course, with its own enrolled students and assigned faculty.
Attendance Session	A single instance of attendance-taking for one section, on one date/period.
Defaulter	A student whose attendance percentage is below the configured shortage threshold.
Correction Request	A student-raised request to amend an attendance record.
RBAC	Role-Based Access Control.
RTM	Requirements Traceability Matrix.
FR / NFR / SEC	Functional Requirement / Non-Functional Requirement / Security Requirement identifiers used throughout this document.

Appendix B: Field Layouts

B.1 Core Data Fields

Entity	Field
User	id, name, email, password_hash, role
Course	id, code, title
Section	id, course_id, code, faculty_id
Timetable	id, section_id, day_of_week, start_time, end_time
AttendanceSession	id, section_id, date, status, created_at, closed_at
AttendanceRecord	id, session_id, student_id, status
CorrectionRequest	id, record_id, student_id, reason, status, acted_by, acted_at
Enrollment	id, section_id, student_id
Report	id, section_id, generated_by, generated_at, type
Config (threshold)	id, course_id, shortage_threshold
AuditLog	id, entity, entity_id, action, actor_id, timestamp
Notification	id, recipient_id, type, sent_at
PasswordResetToken	id, user_id, token, expires_at
Session Token	id, user_id, token, issued_at, expires_at

B.2 Sample Report Requirements

Report	Required Columns
Defaulter Report	Student Name, USN, Course, Attendance %, Threshold, Status
Section Attendance Summary	Section, Date Range, Average Attendance %, Total Sessions

Appendix C: Requirement Traceability Matrix

A representative sample of the Requirement Traceability Matrix is shown below; the full matrix is maintained alongside the codebase and is updated as architecture, design, and test artifacts (see the AttendEase SAD and Software Test Plan) are completed.

Req ID	Architecture	Design	Code	Test Case
FR-001	Auth API/Service (SAD 3.4)	—	TBD	TC-01
SEC-002	Auth API/Service (SAD 3.4, 3.9)	—	TBD	TC-02
FR-013	Academic Setup API/Service (SAD 3.4)	—	TBD	TC-03
FR-014, FR-017	Attendance API/Service (SAD 3.4)	SAD Fig. 2	TBD	TC-04
FR-037	Correction	SAD Fig. 2	TBD	TC-05

Req ID	Architecture	Design	Code	Test Case
	API/Service (SAD 3.4, 3.8); enforced at runtime by Attendance Service			
FR-020	Student Web UI	—	TBD	TC-06
FR-024	Correction API/Service (SAD 3.4)	SAD Fig. 3	TBD	TC-07
FR-027, SEC-006	Correction API/Service (SAD 3.4)	SAD Fig. 3	TBD	TC-08
SEC-003	Cross-cutting RBAC enforcement (SAD 3.8, 3.9)	—	TBD	TC-09
FR-031	Reporting API/Service (SAD 3.4)	—	TBD	TC-10
NFR-001, NFR-002	Attendance API/Service (SAD 3.4)	—	TBD	TC-11
SEC-004	Attendance API/Service (SAD 3.4, 3.8)	—	TBD	TC-13